

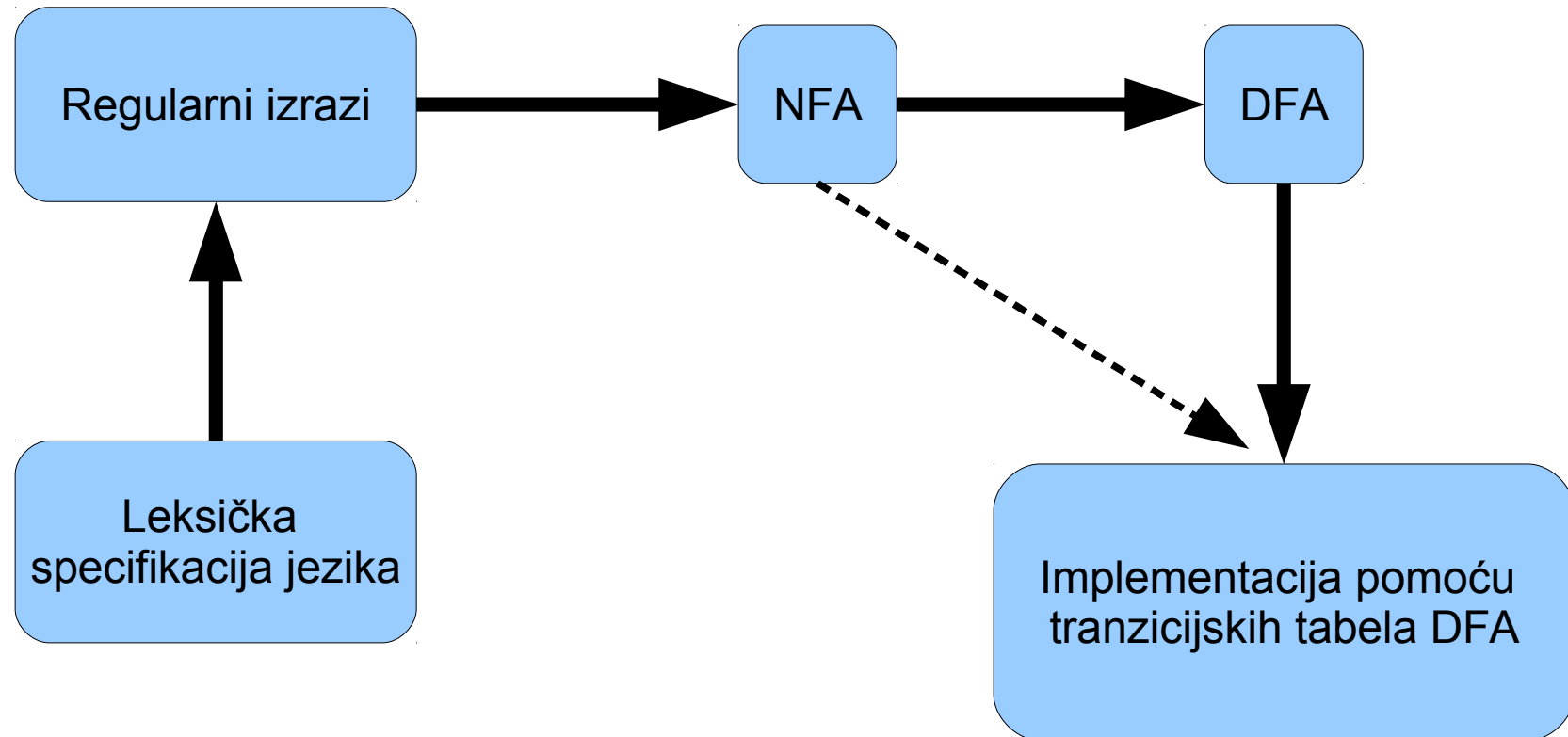
Dizajn kompajlera

dr.sc. Edin Pjanić

Pregled predavanja

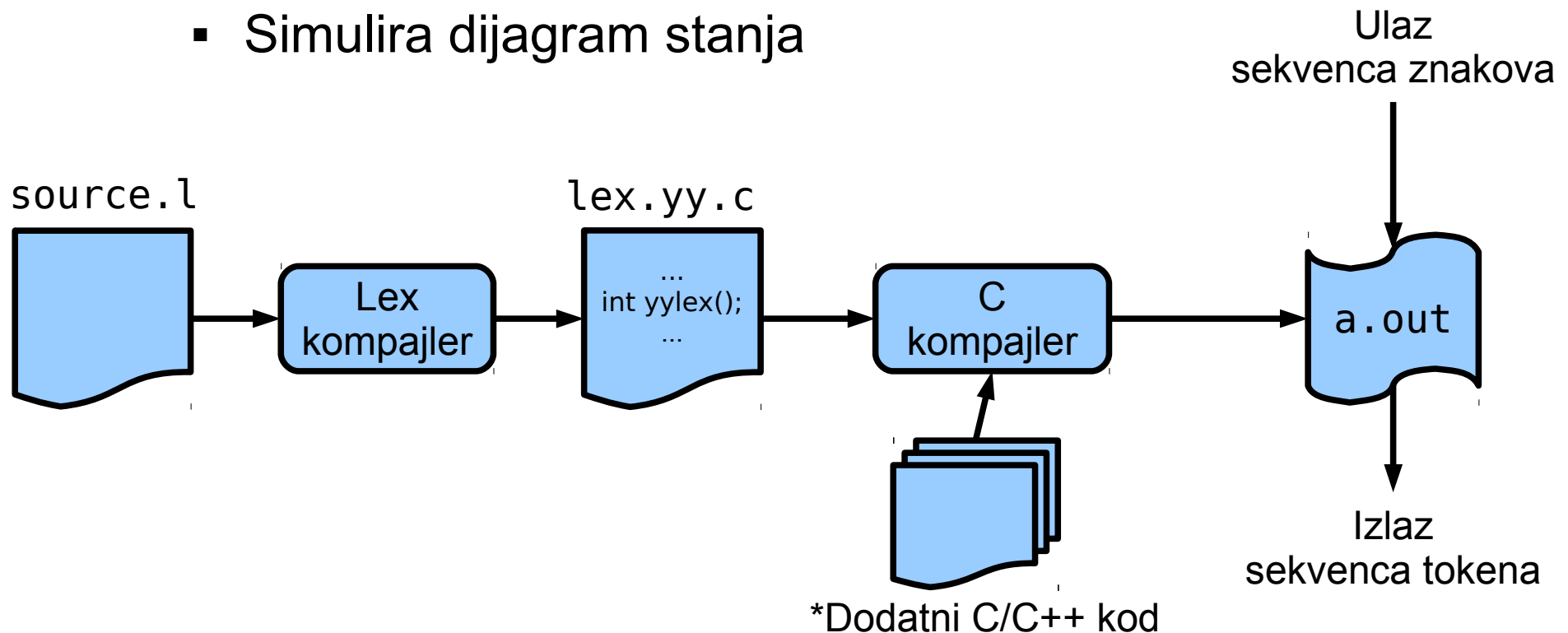
- Lex – alat za generisanje leksera
- Sintaktička analiza (parser)
 - Terminologija i osnovni principi

Implementacija leksera



Lex (flex)

- Alat za generisanje leksičkog analizatora
- Regularni izrazi: opis uzoraka za tokene
 - Transformišu se u dijagram stanja
- Lex kompajler proizvodi `lex.yy.c` fajl
 - Simulira dijagram stanja



Struktura Lex programa

`%{`

C definicije

(ovaj dio se kopira u lex.yy.c)

`%}`

Regex definicije

Deklaracije

`%%`

Tranzicijska pravila

`%%`

Pomoćne C funkcije

Definisanje uzoraka za tokene (regex) i akcije (C kod) koja će se izvršiti u slučaju pronalaska uzorka u ulaznoj sekvenci znakova.

Akcije idu u funkciju **yylex()**.

Format pravila:

uzorak { akcija }

Direktno se kopira u lex.yy.c

Primitivi za definisanje uzorka

Metakarakter	Odgovara stringu
.	bilo koji karakter osim nove linije
\n	nova linija
*	nula ili više kopija prethodnog izraza
+	jedan ili više kopija prethodnog izraza
?	nula ili jedna kopija prethodnog izraza
^	početak linije
\$	kraj linije
a b	a ili b
(ab) +	jedna ili više kopija grupe ab
"a+b"	doslovan izrazu naveden navodnicima
[]	jedan od karaktera navedenih u uglastim zagradama

Izraz	Odgovara stringu
abc	abc
abc*	ab abc abcc abccc ...
abc+	abc abcc abccc ...
a(bc)+	abc abcbc abcbcbc ...
a(bc)?	a abc
[abc]	jedan od: a, b, c
[a-z]	bilo koje slovo od a do z
[a\ -z]	jednan od znakova: a, - ili z
[-az]	jedan od: -, a, z (isto kao prethodni primjer)
[A-Za-z0-9]+	jedan ili više alfanumeričkih znakova
[\t\n]+	praznina
[^ab]	bilo koji karakter osim: a, b
[a^b]	jedan od: a, ^, b
[a b]	jedan od: a, b
a b	jedan od: a, b

Lex globalne varijable

- `char * yytext`
 - Pointer na početak leksema (NULL terminirani C string)
- `int yyleng`
 - Dužina pronađenog leksema
- Procesiranje ulaza:
 1. Analizira se ulaz i traži se string koji odgovara nekom uzorku (najduži pa po prioritetu).
 2. Za pronađeni uzorak, leksem se postavlja dostupnim preko `yytext`, dužina leksema u `yyleng`.
 3. Izvršava se specificirana akcija (ako je zadana).

Primjer – broj znakova, riječi i linija

```
%{  
    int numChars = 0, numWords = 0, numLines = 0;  
}%  
  
%%  
  
\n        {numLines++; numChars++;}  
[^ \t\n]+ {numWords++; numChars += yyleng;}  
.  
        {numChars++;}  
  
%%  
  
int main() {  
    yylex();  
    printf("%d\t%d\t%d\n", numChars, numWords, numLines);  
}
```

Kompajliranje i izvršavanje

```
$ flex brojzrl.l  
$ gcc -o brojanje lex.yy.c -ll  
$ ./brojanje < brojzrl.l  
$ 258 34 16
```

Lex biblioteka: uključuje funkcije yywrap() i main(), ako ih mi ne definišemo.

Broj znakova, riječi i redova u fajlu brojzrl.l

Primjer

- Potrebno je implementirati leksički analizator za jezik koji se sastoji od sljedećih vrsta riječi:
 - Prazna mjesta
 - Ključne riječi: if, then i for
 - Identifikatori (sastoje se od slova engleskog alfabeta ili cifara, pri čemu prvi znak može biti samo slovo)
 - Brojeva (cijeli brojevi)
 - Operator dodjeljivanja =
- Napisati Lex kod za implementaciju leksera!

Kompajler

- Leksička analiza (leksiranje)
 - **Sintaktička analiza (parsiranje, raščlanjivanje)**
 - Semantička analiza
 - Generisanje koda
-
- Parser od leksera uzima sekvencu tokena
 - Cilj: formirati strukturu programa (stablo parsiranja)
 - Regularni izrazi nisu dovoljni za ovaj zadatak
 - Hijerarhijska struktura programa
 - Regularni jezici ne mogu “brojati”. Npr. $\{ \binom{i}{i} \mid i \geq 0 \}$

Parser

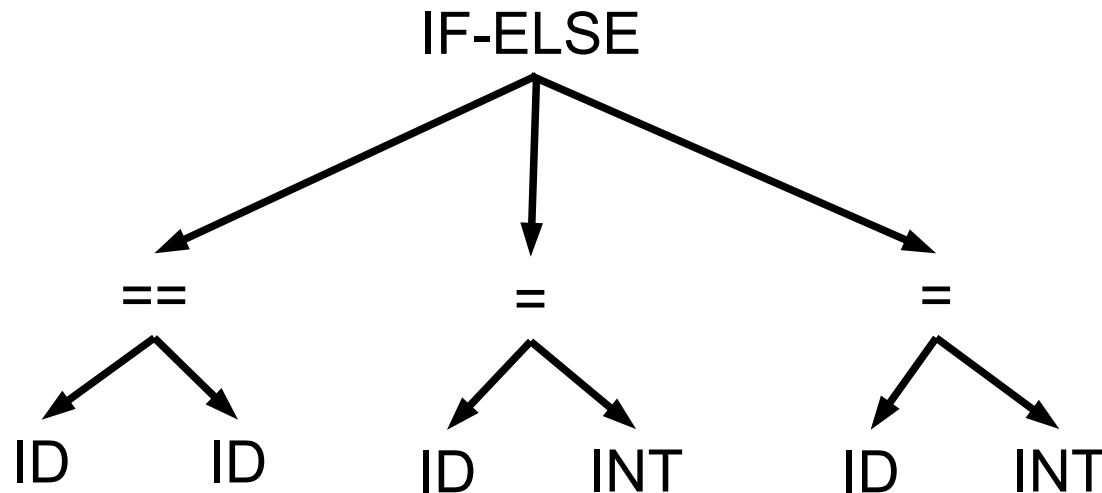
- Primjer:

`if (x == y) a = 1; else a = 2;`

- Ulaz u parser, tj. izlaz iz leksera:

`IF ( ID == ID ) ID = INT ; ELSE ID = INT ;`

- Izlaz iz parsera:



- Potrebno je opisati i prepoznati neispravnu sekvencu tokena.

Kontekstno-slobodna gramatika

(Context-free grammar)

- Programi imaju rekurzivnu strukturu.
- Npr. U C/C++ jeziku izraz EXPR može biti na mjestima :

```
if ( EXPR ) EXPR; else EXPR;  
while (EXPR) EXPR;  
EXPR;  
...
```
- Kontekstno-slobodna gramatika:
 - Način zapisivanja rekurzivne strukture programa.
 - Definisana pravila za izraz važe bez obzira na kontekst.

Kontekstno-slobodna gramatika

- Sastoji se od:
 - Skupa *terminala* T
 - Skupa *neterminala* N
 - Početnog *simbola* $S, S \in N$
 - Skupa *produkcija (produkcijских pravila)*

$$X \rightarrow Y_1 Y_2 \dots Y_n$$

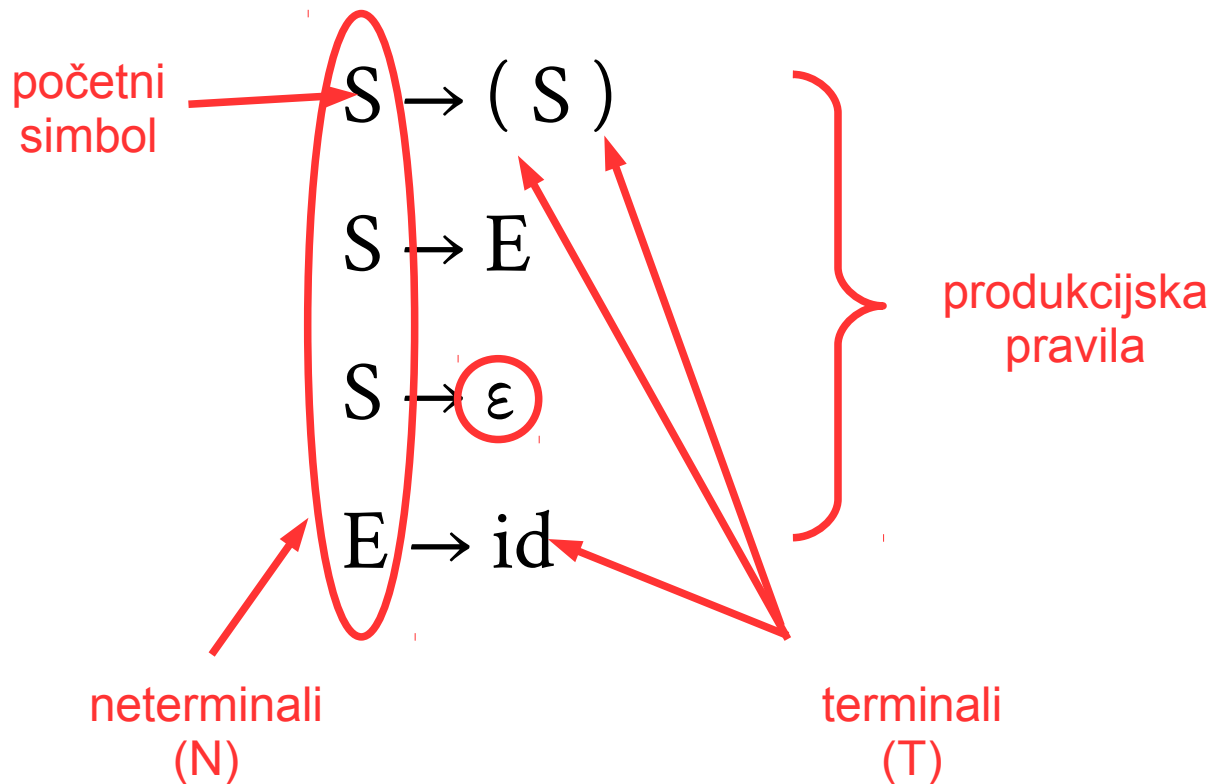
gdje su:

$$X \in N$$

$$Y_i \in N \cup T \cup \{ \varepsilon \}$$

- Neka je $L(G)$ skup svih stringova koji se mogu generisati iz gramatike G
- Ako je G kontekstno-slobodna gramatika onda je $L(G)$ kontekstno-slobodni jezik.

Primjer gramatike (CFG)



Terminali su elementi za koje nema definisanih produkcijskih pravila, obično tokeni.

Derivacije

- Svaka primjena produkcijskog pravila je korak u *derivaciji*.
- Ako gramatika sadrži produkcijsko pravilo:

- $X \rightarrow \gamma$

- Derivacijski korak koji koristi to pravilo se zapisuje kao:

- $\alpha X \beta \rightarrow \alpha \gamma \beta$

- Sekvencu koraka u derivaciji:

$$S \rightarrow (S) \rightarrow (E) \rightarrow (id)$$

možemo zapisati kao:

$$S \rightarrow^* (id)$$

$$\alpha \rightarrow^* \beta \iff \alpha = \beta \text{ ili } \alpha \rightarrow \beta \text{ ili}$$

$$\text{Postoji } \gamma, \alpha \rightarrow \gamma \text{ i } \gamma \rightarrow^* \beta$$

Odnosno, formalno:



Derivacija

- Derivacija je sekvenca koraka koja:
 - Počinje početnim simbolom
 - Završava stringom koji se sastoji samo od terminalnih simbola

$$S \rightarrow^* x_1 x_2 \dots x_n$$

- U svakom koraku derivacije aplicira se jedno produkcijsko pravilo.
- Jezik koji se može generisati takvom gramatikom:

$$L(G) = \{ x_1 x_2 \dots x_n \mid S \rightarrow^* x_1 x_2 \dots x_n \}$$

Primjer

- Neka je zadana gramatika:

$$S \rightarrow E$$

$$E \rightarrow E + E$$

$$| E * E$$

$$| (E)$$

$$| id$$

- Mogući koraci za:

$$S \xrightarrow{*} id + id * id$$

$$S \rightarrow E$$

$$\rightarrow E + E$$

$$\rightarrow id + E$$

$$\rightarrow id + E * E$$

$$\rightarrow id + id * E$$

$$\rightarrow id + id * id$$

Stablo parsiranja

- Derivacija definiše stablo parsiranja.
- Redoslijed dodavanja čvorova je identičan redoslijedu koraka derivacije
- Listovi stabla, čitani slijeva udesno, predstavljaju ulazni string.

$S \rightarrow E$

$\rightarrow E + E$

$\rightarrow id + E$

$\rightarrow id + E * E$

$\rightarrow id + id * E$

$\rightarrow id + id * id$

